
```
inputTextA.draw()  
inputTextB.draw()  
displayTextA.draw()  
displayTextB.draw()  
restartButton.draw()  
checkBoxA.draw()  
checkBoxB.draw()  
radioCustom1.draw()  
radioCustom2.draw()  
radioCustom3.draw()  
checkBoxC.draw()  
radioDefault1.draw()  
radioDefault2.draw()  
radioDefault3.draw()  
statusButton.draw()
```

From the client's point of view, each line just calls the `draw()` method and passes in nothing. From an internal point of view, the code to implement each of these methods is very different. The `draw()` method of the `TextButton` class is completely different from that of the `InputText` class, for instance.

Additionally, all widgets that manage a value contain a `setValue()` and optionally a `getValue()` method. For example, to get the text the user enters into an `InputText` widget, you call the `getValue()` getter method. Radio button and checkbox widgets also have a `getValue()` method to get their current values. To put new text into a `DisplayText` widget, you call the `setValue()` setter method, passing in the new text. Radio button and checkbox widgets can be set with a call to their `setValue()` method.

Polymorphism allows client programmers to feel comfortable with a collection of classes. When clients see a pattern, like using methods named `handleEvent()` and `draw()`, it makes it easy for them to predict how to use a new class in the same collection.

As of this writing, the `pygwidgets` package does not provide either a horizontal or a vertical `Slider` class widget to allow a user to easily select from a range of numbers. If I were to add these widgets, they would certainly contain the following: a `handleEvent()` method, where all user interaction would take place; a `getValue()` and a `setValue()` method to get and set a current value for the `Slider`; and a `draw()` method.